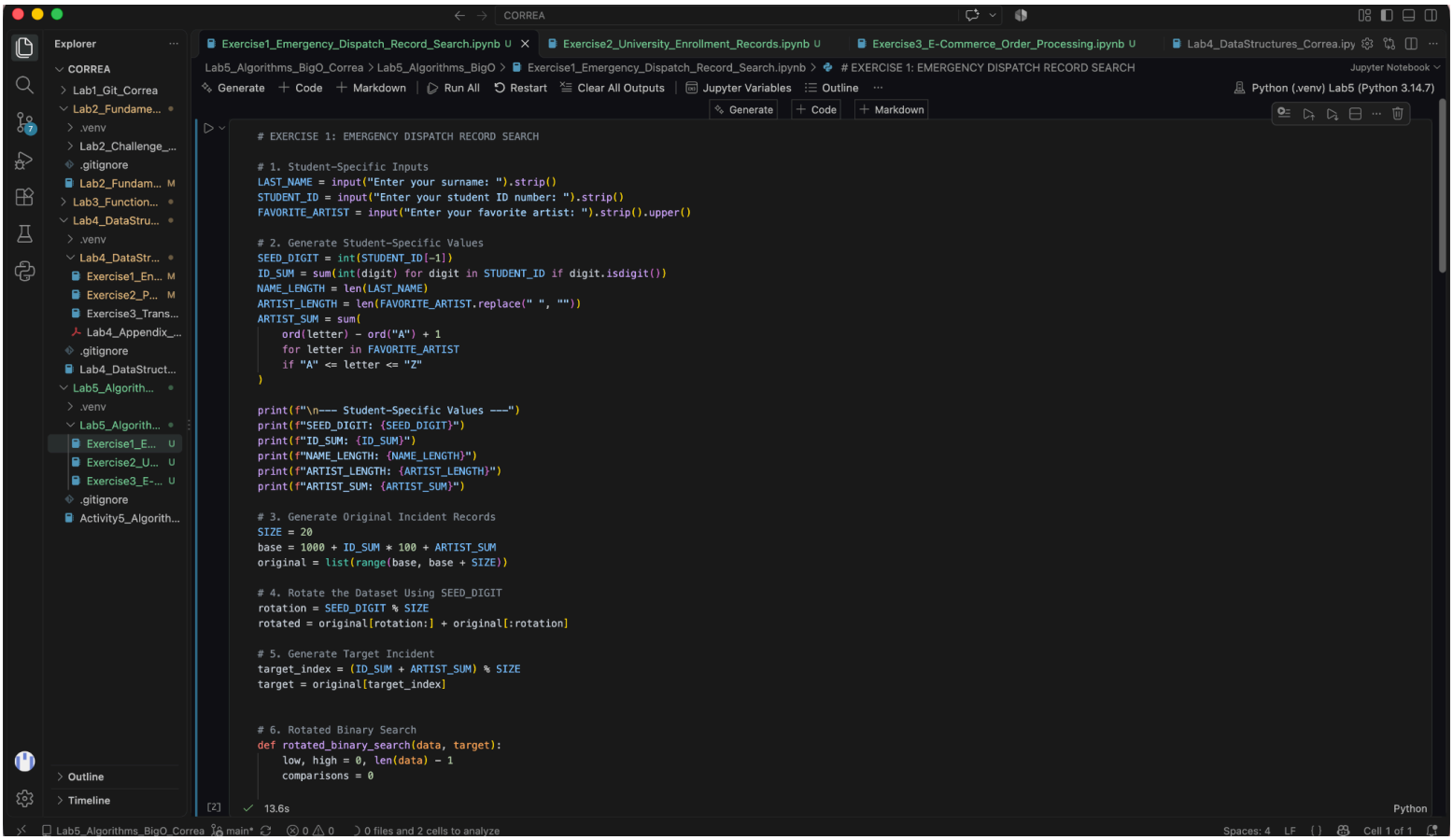
Princess Jherielle M. Correa
BSECE - 1C

October 19, 2026
Computer Programming

Activity 3

FUNCTIONS AND MODULES

EXERCISE #1 EMERGENCY DISPATCH RECORD



```python
# EXERCISE 1: EMERGENCY DISPATCH RECORD SEARCH

# 1. Student-Specific Inputs
LAST_NAME = input("Enter your surname: ").strip()
STUDENT_ID = input("Enter your student ID number: ").strip()
FAVORITE_ARTIST = input("Enter your favorite artist: ").strip().upper()

# 2. Generate Student-Specific Values
SEED_DIGIT = int(STUDENT_ID[-1])
ID_SUM = sum(int(digit) for digit in STUDENT_ID if digit.isdigit())
NAME_LENGTH = len(LAST_NAME)
ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))
ARTIST_SUM = sum(
    ord(letter) - ord("A") + 1
    for letter in FAVORITE_ARTIST
    if "A" <= letter <= "Z"
)

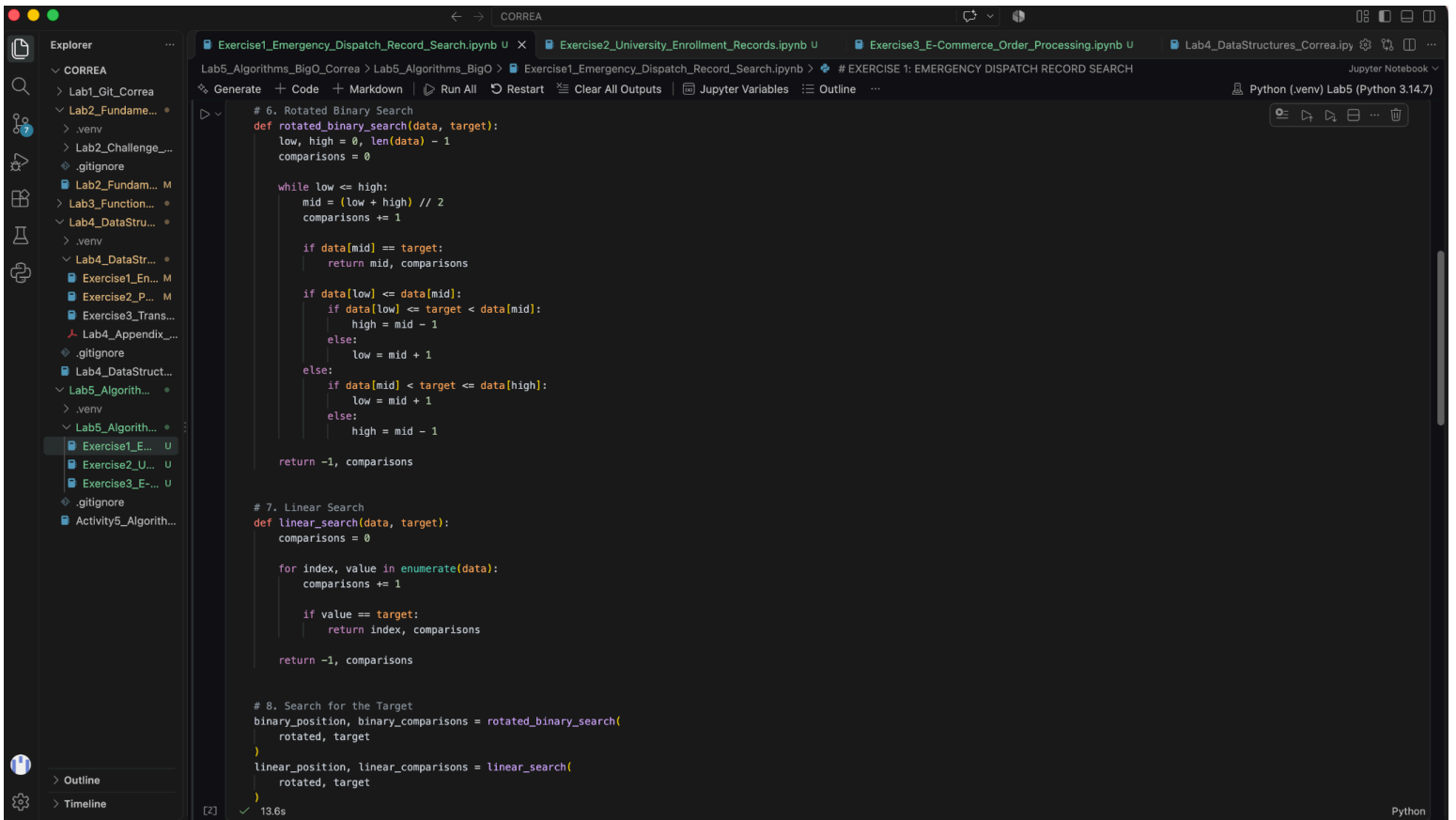
print(f"\n--- Student-Specific Values ---")
print(f"SEED_DIGIT: {SEED_DIGIT}")
print(f"ID_SUM: {ID_SUM}")
print(f"NAME_LENGTH: {NAME_LENGTH}")
print(f"ARTIST_LENGTH: {ARTIST_LENGTH}")
print(f"ARTIST_SUM: {ARTIST_SUM}")

# 3. Generate Original Incident Records
SIZE = 20
base = 1000 + ID_SUM * 100 + ARTIST_SUM
original = list(range(base, base + SIZE))

# 4. Rotate the Dataset Using SEED_DIGIT
rotation = SEED_DIGIT % SIZE
rotated = original[rotation:] + original[:rotation]

# 5. Generate Target Incident
target_index = (ID_SUM + ARTIST_SUM) % SIZE
target = original[target_index]


# 6. Rotated Binary Search
def rotated_binary_search(data, target):
    low, high = 0, len(data) - 1
    comparisons = 0
```



```python
# 6. Rotated Binary Search
def rotated_binary_search(data, target):
    low, high = 0, len(data) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if data[mid] == target:
            return mid, comparisons

        if data[low] <= data[mid]:
            if data[low] <= target < data[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:
            if data[mid] < target <= data[high]:
                low = mid + 1
            else:
                high = mid - 1

    return -1, comparisons

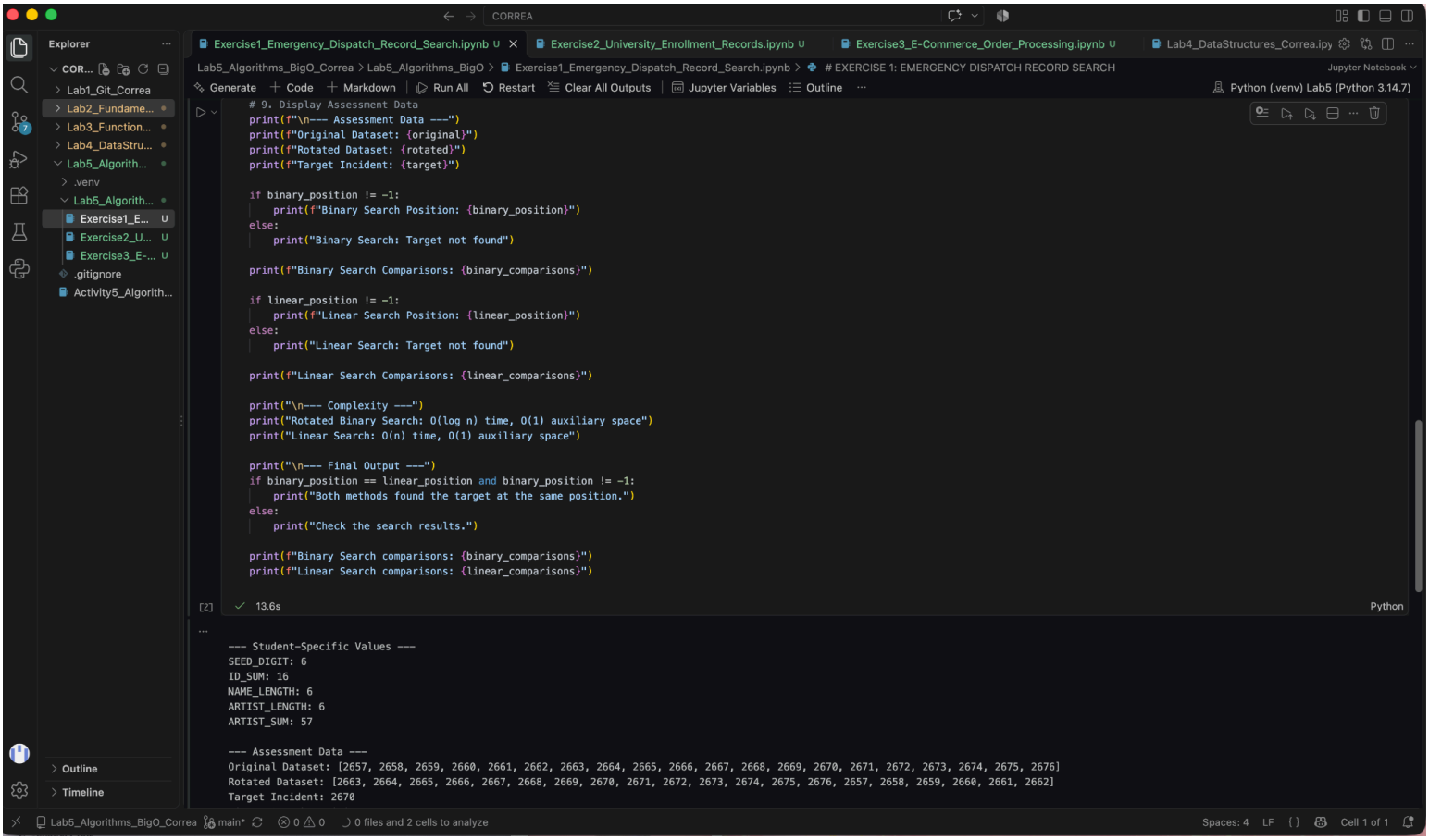

# 7. Linear Search
def linear_search(data, target):
    comparisons = 0

    for index, value in enumerate(data):
        comparisons += 1

        if value == target:
            return index, comparisons

    return -1, comparisons


# 8. Search for the Target
binary_position, binary_comparisons = rotated_binary_search(
    rotated, target
)
linear_position, linear_comparisons = linear_search(
    rotated, target
)
```



ASSESSMENT DATA:

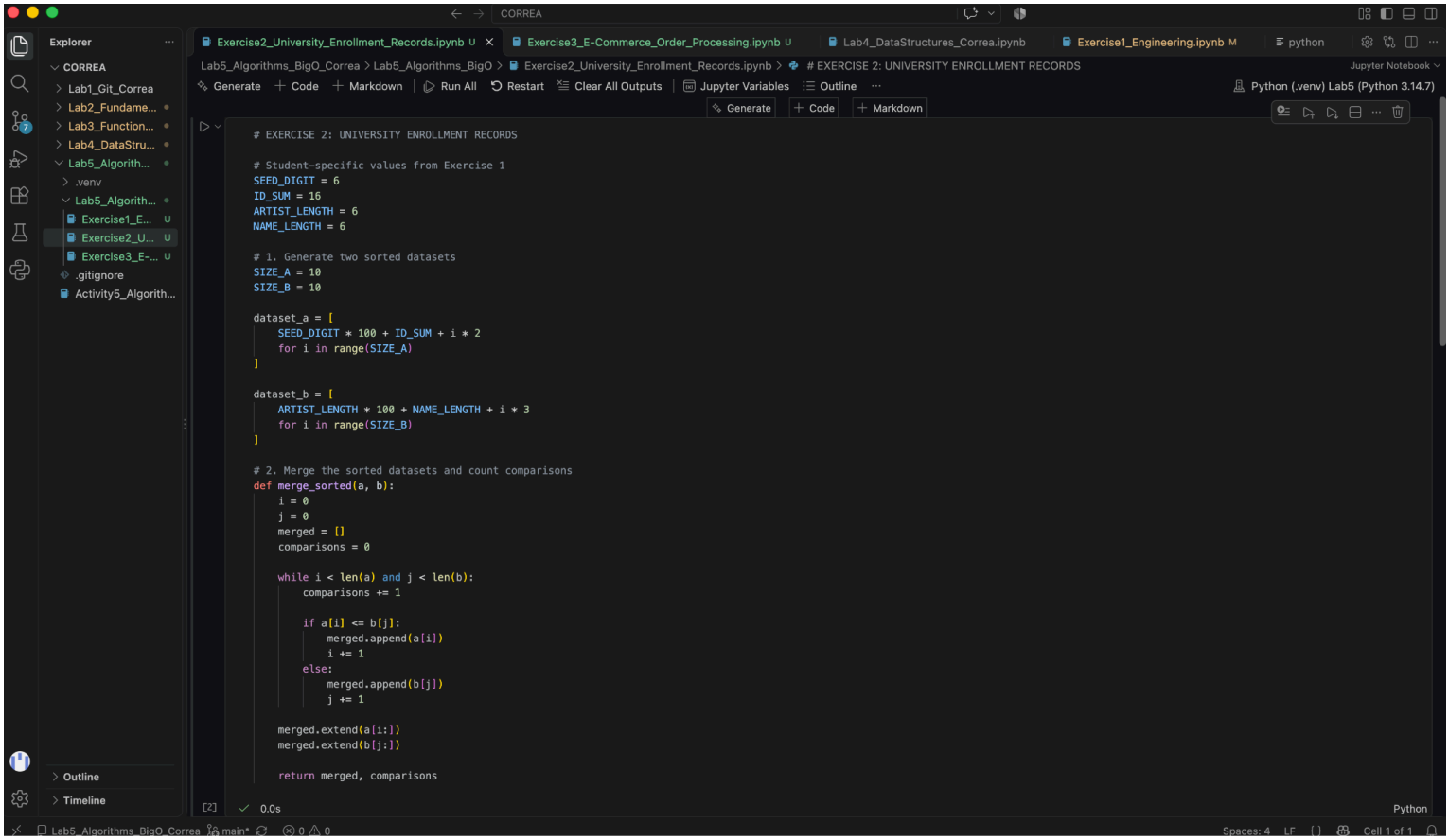
```
--- Student-Specific Values ---
SEED_DIGIT: 6
ID_SUM: 16
NAME_LENGTH: 6
ARTIST_LENGTH: 6
ARTIST_SUM: 57

--- Assessment Data ---
Original Dataset: [2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676]
Rotated Dataset: [2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2657, 2658, 2659, 2660, 2661, 2662]
Target Incident: 2670
Binary Search Position: 7
Binary Search Comparisons: 4
Linear Search Position: 7
Linear Search Comparisons: 8

--- Complexity ---
Rotated Binary Search: O(log n) time, O(1) auxiliary space
Linear Search: O(n) time, O(1) auxiliary space

--- Final Output ---
Both methods found the target at the same position.
Binary Search comparisons: 4
Linear Search comparisons: 8
```

EXERCISE #2 UNIVERSITY ENROLLMENT RECORDS



```python
# EXERCISE 2: UNIVERSITY ENROLLMENT RECORDS

# Student-specific values from Exercise 1
SEED_DIGIT = 6
ID_SUM = 16
ARTIST_LENGTH = 6
NAME_LENGTH = 6

# 1. Generate two sorted datasets
SIZE_A = 10
SIZE_B = 10

dataset_a = [
    SEED_DIGIT * 100 + ID_SUM + i * 2
    for i in range(SIZE_A)
]

dataset_b = [
    ARTIST_LENGTH * 100 + NAME_LENGTH + i * 3
    for i in range(SIZE_B)
]

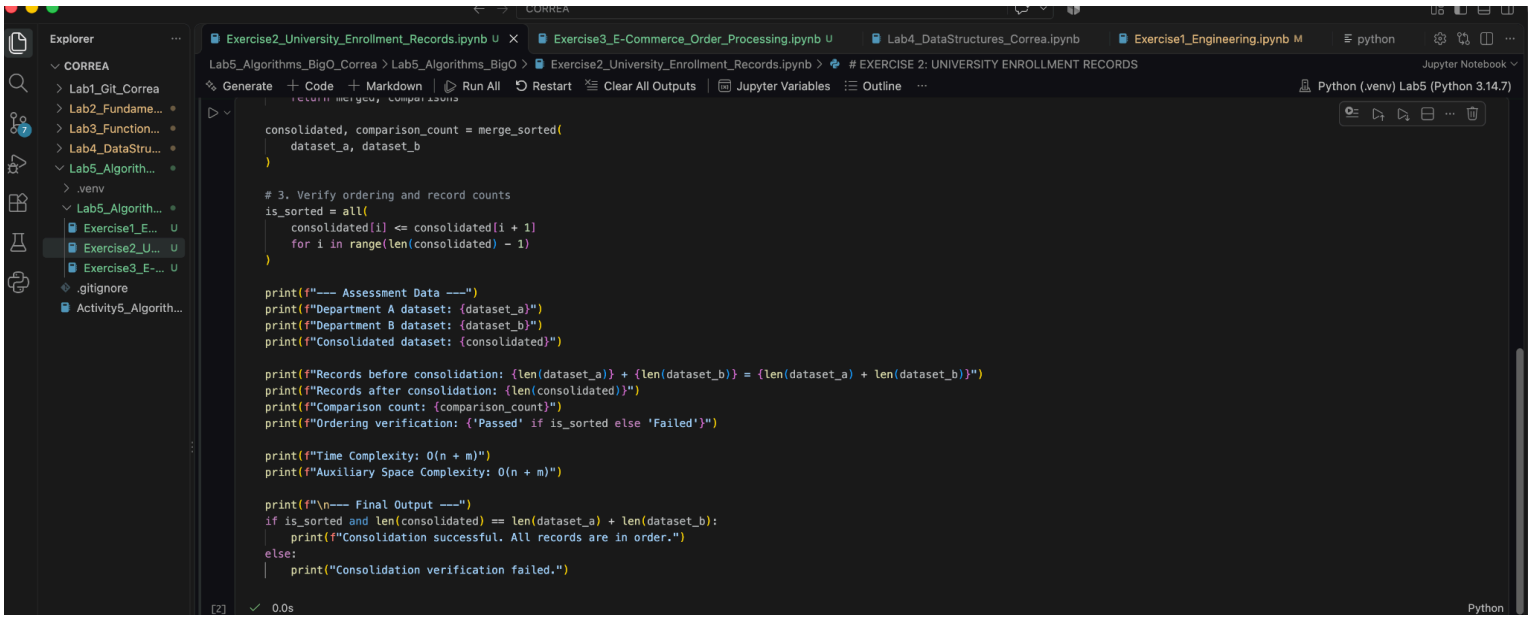
# 2. Merge the sorted datasets and count comparisons
def merge_sorted(a, b):
    i = 0
    j = 0
    merged = []
    comparisons = 0

    while i < len(a) and j < len(b):
        comparisons += 1

        if a[i] <= b[j]:
            merged.append(a[i])
            i += 1
        else:
            merged.append(b[j])
            j += 1

    merged.extend(a[i:])
    merged.extend(b[j:])

    return merged, comparisons
```



```python
consolidated, comparison_count = merge_sorted(
    dataset_a, dataset_b
)

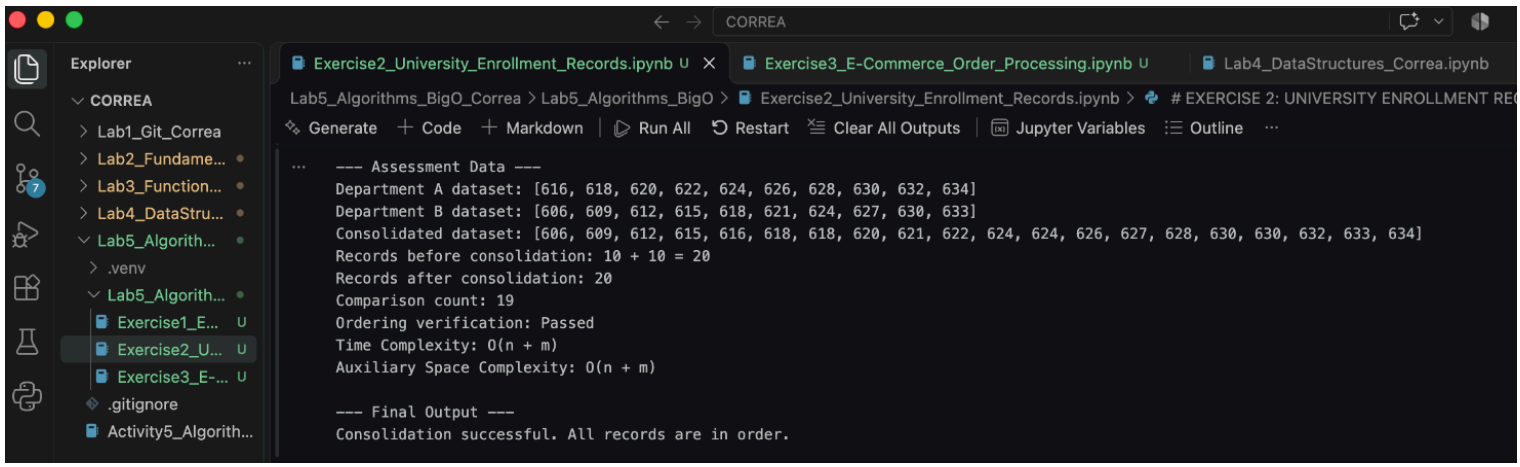
# 3. Verify ordering and record counts
is_sorted = all(
    consolidated[i] <= consolidated[i + 1]
    for i in range(len(consolidated) - 1)
)

print(f"--- Assessment Data ---")
print(f"Department A dataset: {dataset_a}")
print(f"Department B dataset: {dataset_b}")
print(f"Consolidated dataset: {consolidated}")

print(f"Records before consolidation: {len(dataset_a)} + {len(dataset_b)} = {len(dataset_a) + len(dataset_b)}")
print(f"Records after consolidation: {len(consolidated)}")
print(f"Comparison count: {comparison_count}")
print(f"Ordering verification: {'Passed' if is_sorted else 'Failed'}")

print(f"Time Complexity: O(n + m)")
print(f"Auxiliary Space Complexity: O(n + m)")

print(f"\n--- Final Output ---")
if is_sorted and len(consolidated) == len(dataset_a) + len(dataset_b):
    print(f"Consolidation successful. All records are in order.")
else:
    print("Consolidation verification failed.")
```



```
--- Assessment Data ---
Department A dataset: [616, 618, 620, 622, 624, 626, 628, 630, 632, 634]
Department B dataset: [606, 609, 612, 615, 618, 621, 624, 627, 630, 633]
Consolidated dataset: [606, 609, 612, 615, 616, 618, 618, 620, 621, 622, 624, 624, 626, 627, 628, 630, 630, 632, 633, 634]
Records before consolidation: 10 + 10 = 20
Records after consolidation: 20
Comparison count: 19
Ordering verification: Passed
Time Complexity: O(n + m)
Auxiliary Space Complexity: O(n + m)

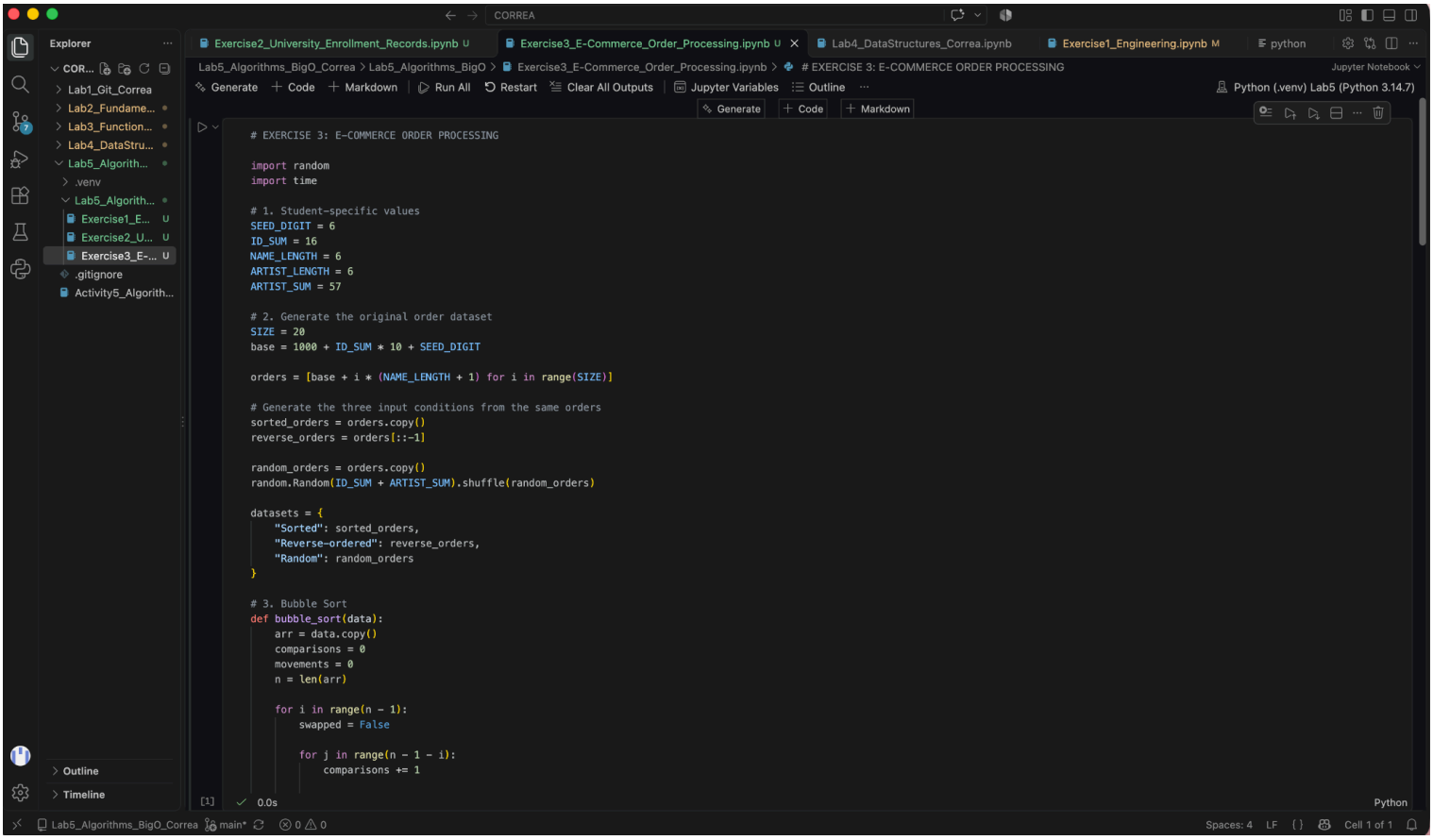
--- Final Output ---
Consolidation successful. All records are in order.
```

ASSESSMENT DATA:

```
--- Assessment Data ---
Department A dataset: [616, 618, 620, 622, 624, 626, 628, 630, 632, 634]
Department B dataset: [606, 609, 612, 615, 618, 621, 624, 627, 630, 633]
Consolidated dataset: [606, 609, 612, 615, 616, 618, 618, 620, 621, 622, 624, 624, 626, 627, 628, 630, 630, 632, 633, 634]
Records before consolidation: 10 + 10 = 20
Records after consolidation: 20
Comparison count: 19
Ordering verification: Passed
Time Complexity: O(n + m)
Auxiliary Space Complexity: O(n + m)

--- Final Output ---
Consolidation successful. All records are in order.
```

EXERCISE # 3 TRANSACTION HISTORY ANALYZER



```python
# EXERCISE 3: E-COMMERCE ORDER PROCESSING

import random
import time

# 1. Student-specific values
SEED_DIGIT = 6
ID_SUM = 16
NAME_LENGTH = 6
ARTIST_LENGTH = 6
ARTIST_SUM = 57

# 2. Generate the original order dataset
SIZE = 20
base = 1000 + ID_SUM * 10 + SEED_DIGIT

orders = [base + i * (NAME_LENGTH + 1) for i in range(SIZE)]

# Generate the three input conditions from the same orders
sorted_orders = orders.copy()
reverse_orders = orders[::-1]

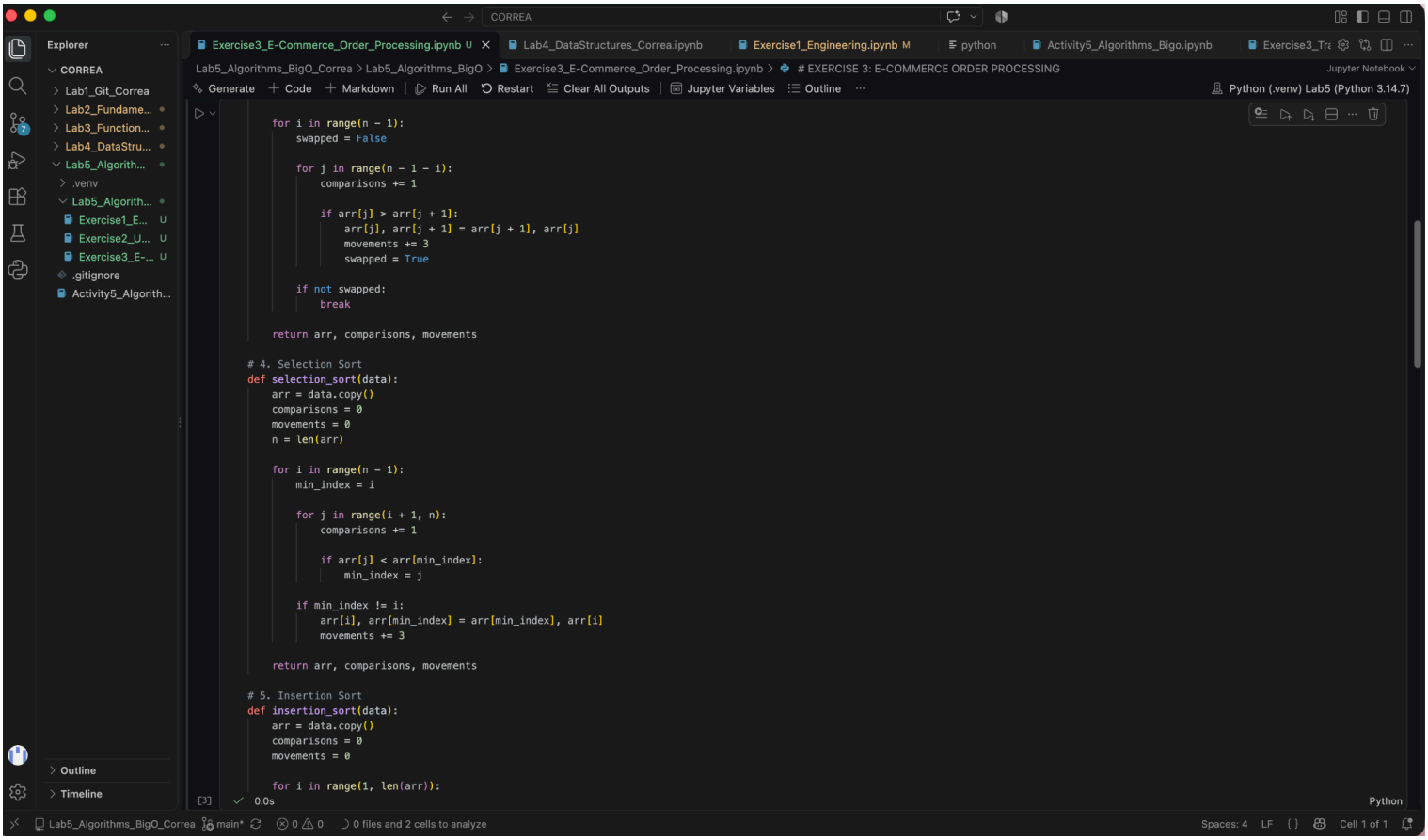
random_orders = orders.copy()
random.Random(ID_SUM + ARTIST_SUM).shuffle(random_orders)

datasets = {
    "Sorted": sorted_orders,
    "Reverse-ordered": reverse_orders,
    "Random": random_orders
}

# 3. Bubble Sort
def bubble_sort(data):
    arr = data.copy()
    comparisons = 0
    movements = 0
    n = len(arr)

    for i in range(n - 1):
        swapped = False

        for j in range(n - 1 - i):
            comparisons += 1
```



```python
        for i in range(n - 1):
            swapped = False

            for j in range(n - 1 - i):
                comparisons += 1

                if arr[j] > arr[j + 1]:
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
                    movements += 3
                    swapped = True

            if not swapped:
                break

        return arr, comparisons, movements

    # 4. Selection Sort
    def selection_sort(data):
        arr = data.copy()
        comparisons = 0
        movements = 0
        n = len(arr)

        for i in range(n - 1):
            min_index = i

            for j in range(i + 1, n):
                comparisons += 1

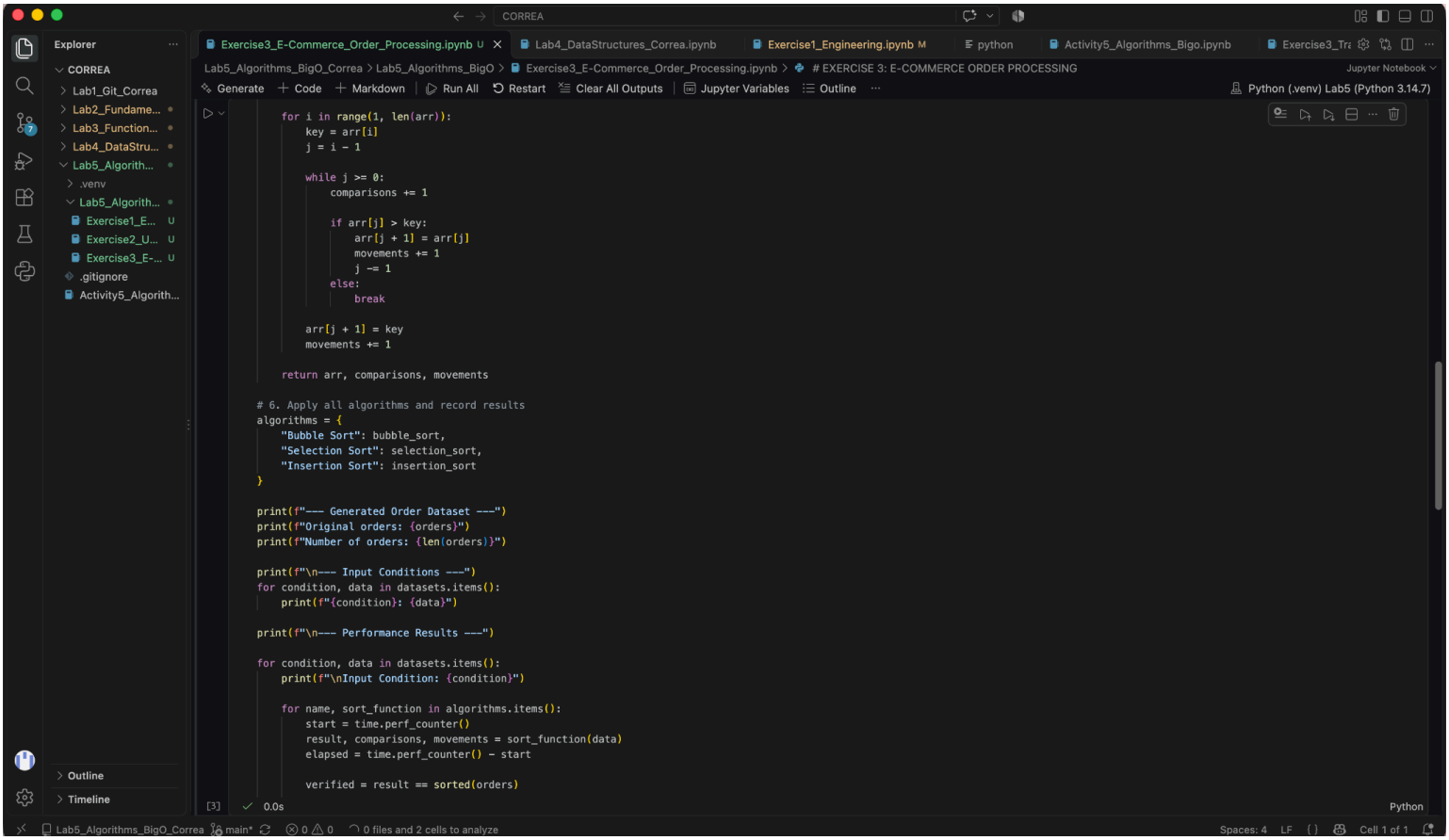
                if arr[j] < arr[min_index]:
                    min_index = j

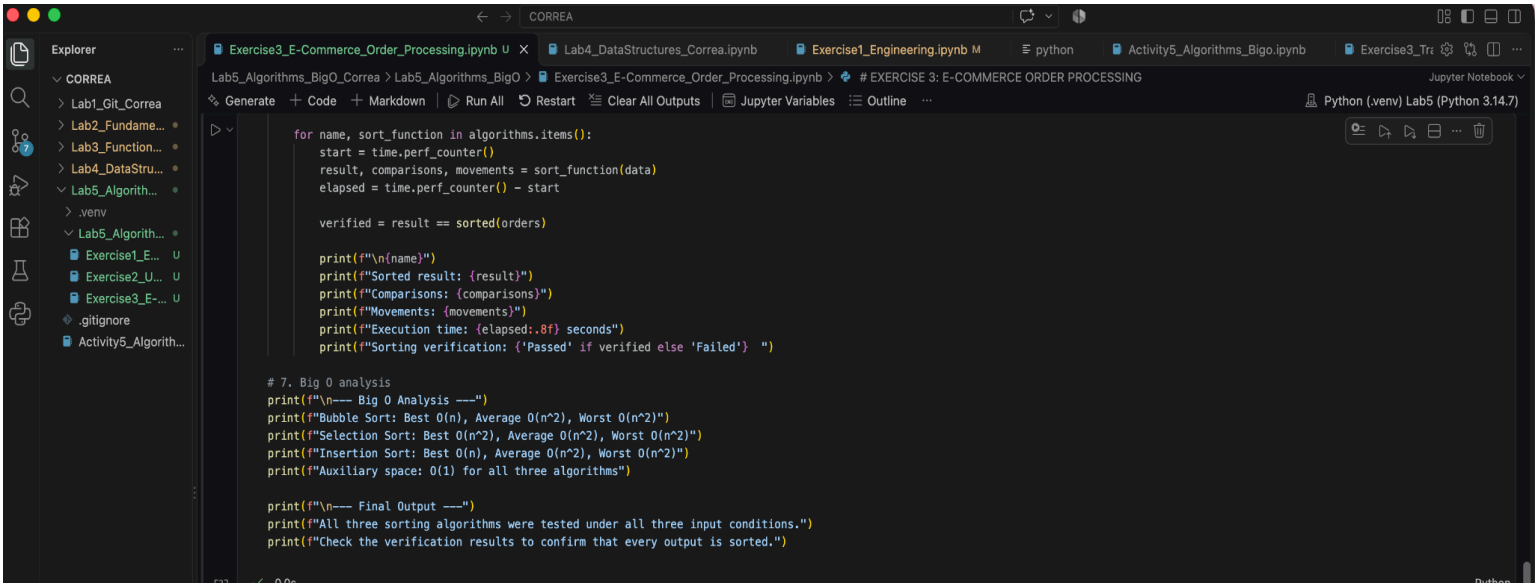
            if min_index != i:
                arr[i], arr[min_index] = arr[min_index], arr[i]
                movements += 3

        return arr, comparisons, movements

    # 5. Insertion Sort
    def insertion_sort(data):
        arr = data.copy()
        comparisons = 0
        movements = 0

        for i in range(1, len(arr)):
```


Exercise3_E-Commerce_Order_Processing.ipynb U
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb M
python
Activity5_Algorithms_Bigo.ipynb
Lab5_Algorithms_BigO_Correa > Lab5_Algorithms_BigO > Exercise3_E-Commerce_Order_Processing.ipynb > # EXERCISE 3: E-COMMERCE ORDER PROCESSING
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python (.venv) Lab5 (Python 3.14.7)
for i in range(1, len(arr)):
key = arr[i]
j = i - 1
while j >= 0:
comparisons += 1
if arr[j] > key:
arr[j + 1] = arr[j]
movements += 1
j -= 1
else:
break
arr[j + 1] = key
movements += 1
return arr, comparisons, movements
6. Apply all algorithms and record results
algorithms = {
"Bubble Sort": bubble_sort,
"Selection Sort": selection_sort,
"Insertion Sort": insertion_sort
}
print(f"--- Generated Order Dataset ---")
print(f"Original orders: {orders}")
print(f"Number of orders: {len(orders)}")
print(f"\n--- Input Conditions ---")
for condition, data in datasets.items():
print(f"{condition}: {data}")
print(f"\n--- Performance Results ---")
for condition, data in datasets.items():
print(f"\nInput Condition: {condition}")
for name, sort_function in algorithms.items():
start = time.perf_counter()
result, comparisons, movements = sort_function(data)
elapsed = time.perf_counter() - start
verified = result == sorted(orders)
[3]
0.0s
Python
Explorer
CORREA
Lab1_Git_Correa
Lab2_Fundame...
Lab3_Function...
Lab4_DataStru...
Lab5_Algorith...
.venv
Lab5_Algorith...
Exercise1_E...
Exercise2_U...
Exercise3_E-...
.gitignore
Activity5_Algorith...
Outline
Timeline
Lab5_Algorithms_BigO_Correa
main*
0 files and 2 cells to analyze
Spaces: 4
LF
Cell 1 of 1


Exercise3_E-Commerce_Order_Processing.ipynb U
Lab4_DataStructures_Correa.ipynb
Exercise1_Engineering.ipynb M
python
Activity5_Algorithms_Bigo.ipynb
Lab5_Algorithms_BigO_Correa > Lab5_Algorithms_BigO > Exercise3_E-Commerce_Order_Processing.ipynb > # EXERCISE 3: E-COMMERCE ORDER PROCESSING
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python (.venv) Lab5 (Python 3.14.7)
for name, sort_function in algorithms.items():
start = time.perf_counter()
result, comparisons, movements = sort_function(data)
elapsed = time.perf_counter() - start
verified = result == sorted(orders)
print(f"\n{name}")
print(f"Sorted result: {result}")
print(f"Comparisons: {comparisons}")
print(f"Movements: {movements}")
print(f"Execution time: {elapsed:.8f} seconds")
print(f"Sorting verification: {'Passed' if verified else 'Failed'} ")
7. Big O analysis
print(f"\n--- Big O Analysis ---")
print(f"Bubble Sort: Best O(n), Average O(n^2), Worst O(n^2)")
print(f"Selection Sort: Best O(n^2), Average O(n^2), Worst O(n^2)")
print(f"Insertion Sort: Best O(n), Average O(n^2), Worst O(n^2)")
print(f"Auxiliary space: O(1) for all three algorithms")
print(f"\n--- Final Output ---")
print(f"All three sorting algorithms were tested under all three input conditions.")
print(f"Check the verification results to confirm that every output is sorted.")
Explorer
CORREA
Lab1_Git_Correa
Lab2_Fundame...
Lab3_Function...
Lab4_DataStru...
Lab5_Algorith...
.venv
Lab5_Algorith...
Exercise1_E...
Exercise2_U...
Exercise3_E-...
.gitignore
Activity5_Algorith...

ASSESSMENT DATA:

```
--- Generated Order Dataset ---
Original orders: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Number of orders: 20

--- Input Conditions ---
Sorted: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Reverse-ordered: [1299, 1292, 1285, 1278, 1271, 1264, 1257, 1250, 1243, 1236, 1229, 1222, 1215, 1208, 1201, 1194, 1187, 1180, 1173, 1166]
Random: [1250, 1257, 1243, 1180, 1285, 1166, 1236, 1292, 1299, 1208, 1264, 1173, 1229, 1194, 1215, 1201, 1271, 1278, 1187, 1222]

--- Performance Results ---

Input Condition: Sorted

Bubble Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 19
Movements: 0
Execution time: 0.00000562 seconds
Sorting verification: Passed

Selection Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 190
Movements: 0
Execution time: 0.00001083 seconds
Sorting verification: Passed

Insertion Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 19
```

```
Movements: 19
Execution time: 0.00000446 seconds
Sorting verification: Passed

Input Condition: Reverse-ordered

Bubble Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 190
Movements: 570
Execution time: 0.00002075 seconds
Sorting verification: Passed

Selection Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 190
Movements: 30
Execution time: 0.00001012 seconds
Sorting verification: Passed

Insertion Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 190
Movements: 209
Execution time: 0.00001471 seconds
Sorting verification: Passed

Input Condition: Random
```

```
Bubble Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 184
Movements: 312
Execution time: 0.00001592 seconds
Sorting verification: Passed

Selection Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 190
Movements: 54
Execution time: 0.00001037 seconds
Sorting verification: Passed

Insertion Sort
Sorted result: [1166, 1173, 1180, 1187, 1194, 1201, 1208, 1215, 1222, 1229, 1236, 1243, 1250, 1257, 1264, 1271, 1278, 1285, 1292, 1299]
Comparisons: 120
Movements: 123
Execution time: 0.00000904 seconds
Sorting verification: Passed

--- Big O Analysis ---
Bubble Sort: Best O(n), Average O(n^2), Worst O(n^2)
Selection Sort: Best O(n^2), Average O(n^2), Worst O(n^2)
Insertion Sort: Best O(n), Average O(n^2), Worst O(n^2)
Auxiliary space: O(1) for all three algorithms

--- Final Output ---
All three sorting algorithms were tested under all three input conditions.
Check the verification results to confirm that every output is sorted.
```